

CVP Ops Console Portable README

1. 문서 목적

이 문서는 portable 배포 폴더 안에 들어 있는 **CVP Ops Console** 프로그램의 목적, 구조, 실행 방식, 내부 동작, 저장 방식, 화면 구성, API, 한계, 유지보수 포인트를 가능한 한 빠짐없이 설명하기 위한 기술 설명서입니다.

이 프로그램은 **CVP API를 이용해 CVP에 등록된 장비들의 현황과 정보를 읽어 와서, 사용자가 단순 조회와 확인을 수행할 수 있도록 돕는 읽기 전용 현황 관리 프로그램**입니다.

중요 전제

- 이 프로그램은 CVP를 대체하지 않습니다.
- 이 프로그램은 장비 설정을 변경하지 않습니다.
- 이 프로그램은 Change Control, Provisioning, Config Push 같은 제어 기능을 수행하지 않습니다.
- 이 프로그램은 현재 시점의 스냅샷을 수집하고, 그 결과를 조회 가능한 형태로 정리해 주는 보조 도구입니다.
- 이 프로그램의 판단 결과는 운영 판단 보조가 목적이며, 자동 승인 시스템이 아닙니다.

2. 프로그램이 하는 일

이 프로그램은 크게 다음 일을 합니다.

- CVP 또는 데모 샘플 데이터에서 장비 정보를 수집합니다.
- 장비별 VRF, BGP AS, VLAN, Config 정보를 수집합니다.
- 수집한 Config에서 IP 주소 정보를 파싱합니다.
- 수집 결과를 SQLite DB에 최신 스냅샷 형태로 저장합니다.
- Config 본문은 파일로 따로 저장합니다.
- 웹 UI를 통해 장비, IP, BGP, VLAN, VRF, Config를 조회할 수 있게 합니다.
- 사용자가 신규 IP, VLAN, BGP ASN, VRF를 쓰기 전에 현재 사용 여부를 조회할 수 있게 돕습니다.

3. 이 portable 폴더가 왜 설치 없이 동작하는가

이 폴더는 일반적인 소스코드 배포본이 아니라 **실행 가능한 런타임 묶음**입니다.

즉, 아래 요소가 이미 포함되어 있습니다.

- Python 런타임 자체
- Python 표준 라이브러리
- FastAPI, Uvicorn, requests, grpcio 등 필요한 Python 패키지

- backend 소스코드
- build 완료된 frontend 정적 파일
- cloudvision-python-trunk 라이브러리
- 실행용 배치 파일
- 설정 파일

그래서 대상 PC에서 별도로 설치할 것은 없습니다.

- Python 설치 불필요
- npm 설치 불필요
- venv 생성 불필요
- pip install 불필요

4. portable 배포 폴더 구조

현재 portable 폴더의 최상위 구조는 다음과 같습니다.

```
cvp-ops-console-portable/
├─ backend/
├─ cloudvision-python-trunk/
├─ config/
├─ frontend/
├─ python/
├─ run-live.bat
├─ run-demo.bat
├─ README.md
└─ README.txt
```

최상위 구성 설명

- backend/
 - 실제 백엔드 애플리케이션 코드
 - API, 수집, 저장, 조회 로직 포함
- cloudvision-python-trunk/
 - Arista CloudVision Python 라이브러리 원본
 - CVP gRPC 질의에 사용
- config/
 - 실행 시 읽는 환경설정 파일 보관
 - live.env 포함
- frontend/
 - build 완료된 웹 UI 파일 보관

- frontend/dist 내부에 index.html, assets 포함
- python/
 - portable Python 런타임 전체
- run-live.bat
 - 실제 CVP에 연결하는 실행기
- run-demo.bat
 - 샘플 데이터로 실행하는 실행기
- README.md
 - 현재 Markdown 설명서
- README.txt
 - 텍스트 버전 설명서

5. 폴더별 상세 설명

5-1. backend/

이 폴더는 서버 프로그램의 핵심입니다. FastAPI 애플리케이션이 여기서 실행됩니다.

세부 구성:

- backend/app/
 - 실제 Python 코드
- backend/config/
 - 텔레메트리 경로와 필드 매핑 YAML
- backend/data/
 - 실행 중 생성되거나 참조되는 데이터 저장 폴더

5-2. backend/data/

이 폴더는 실행 중 데이터 저장소 역할을 합니다.

세부 구성:

- backend/data/db/
 - SQLite DB 파일 저장 위치
 - 기본 DB 파일명은 ops_console.db
- backend/data/configs/
 - 장비별 running-config 파일 저장 위치
- backend/data/sample_snapshot.json
 - demo 모드에서 읽는 샘플 데이터

5-3. cloudvision-python-trunk/

이 폴더는 CVP와 gRPC로 통신하기 위해 함께 포함된 라이브러리입니다.

백엔드는 이 폴더를 `sys.path`에 넣고 `cloudvision.Connector.grpc_client`의 `GRPCClient`를 import하여 사용합니다.

이 폴더를 함께 넣는 이유는 다음과 같습니다.

- 대상 PC에 별도 패키지 설치를 하지 않기 위해
- 기존 `cloudvision-python-trunk` 라이브러리를 그대로 재사용하기 위해
- CVP의 `dataset/path` 기반 조회를 안정적으로 수행하기 위해

5-4. `config/live.env`

이 파일은 실제 CVP 연결용 설정 파일입니다. `run-live.bat`가 이 파일을 읽어 환경변수로 설정합니다.

주요 항목

- `CVP_HOST`
 - CVP IP 또는 호스트명
- `CVP_PORT`
 - 기본 443
- `CVP_USERNAME`
 - CVP 로그인 계정
- `CVP_PASSWORD`
 - CVP 비밀번호
- `CVP_INSECURE_TLS`
 - `true`이면 인증서 검증을 완화한 접속 허용
- `CVP_DEVICE_IDS`
 - 특정 장비만 수집하고 싶을 때 device id를 쉼표로 나열
- `OPS_CONSOLE_BIND_HOST`
 - 웹 서버 바인드 주소
- `OPS_CONSOLE_BIND_PORT`
 - 웹 서버 포트

보안 주의사항

- 이 파일은 평문입니다.
- 즉, 비밀번호가 그대로 저장됩니다.
- 배포 시 이 파일 접근 권한을 제한하는 것이 좋습니다.
- 외부 반출본을 만들 때는 실제 계정정보를 제거한 뒤 배포하는 것이 좋습니다.

5-5. python/

이 폴더는 portable Python 런타임입니다. 이 안에 `python.exe` 와 표준 라이브러리, DLL, `site-packages` 가 모두 있습니다.

실행 시 `run-live.bat` 와 `run-demo.bat` 는 시스템 Python을 쓰지 않고 아래 실행 파일을 직접 호출합니다.

```
python\python.exe
```

5-6. frontend/dist/

이 폴더는 React/Vite 프론트엔드를 build한 결과물입니다. 즉, 개발용 소스가 아니라 브라우저가 실제 읽는 정적 파일입니다.

포함 요소:

- `index.html`
- `assets/*.js`
- `assets/*.css`
- favicon, icon 파일

6. 실행 방식

6-1. run-live.bat 실행 흐름

`run-live.bat` 는 다음 순서로 동작합니다.

- 현재 폴더 경로를 `ROOT` 로 잡습니다.
- `config\live.env` 를 읽습니다.
- 파일의 각 `key=value` 를 환경변수로 설정합니다.
- `OPS_CONSOLE_USE MOCK=false` 로 설정합니다.
- `OPS_CONSOLE_CVP_LIBRARY_ROOT` 를 `ROOT\cloudvision-python-trunk` 로 설정합니다.
- `ROOT\python\python.exe` 를 사용해 `uvicorn` 을 실행합니다.
- `uvicorn` 이 `backend/app/main.py` 의 `app` 객체를 띄웁니다.

즉, `run-live.bat` 는 **실CVP 연결 모드 서버 실행기**입니다.

6-2. run-demo.bat 실행 흐름

`run-demo.bat` 는 다음 순서로 동작합니다.

- `OPS_CONSOLE_USE MOCK=true` 로 설정합니다.

- `cloudvision` 라이브러리 경로를 설정합니다.
- `python\python.exe` 로 `uvicorn` 을 실행합니다.
- demo 모드에서는 `sample_snapshot.json` 을 읽습니다.

즉, `run-demo.bat` 는 샘플 데이터 확인용 실행기입니다.

7. 서버 기동 후 내부에서 일어나는 일

서버가 뜨면 FastAPI 애플리케이션이 초기화됩니다. 핵심 시작점은 `backend/app/main.py` 입니다.

`main.py` 의 역할

- `Settings` 로드
- `SnapshotRepository` 생성
- `ConfigFileManager` 생성
- `CollectionService` 생성
- `QueryService` 생성
- API Router 등록
- startup 이벤트에서 초기 데이터 보장
- `health` endpoint 제공
- `frontend/dist` 를 정적 파일로 서비스
- `/` 요청 시 `index.html` 응답

즉, 백엔드 서버 하나가 아래 두 역할을 동시에 합니다.

- REST API 서버 역할
- 정적 프론트 파일 웹서버 역할

8. 설정 계층 상세 설명

설정은 `backend/app/core/settings.py` 에서 읽습니다.

`Settings` 가 결정하는 값

- `backend_dir`
- `console_dir`
- `project_root`
- `db_path`
- `config_dir`
- `sample_snapshot_path`
- `telemetry_paths_path`
- `field_mapping_path`

- `cvp_library_root`
- `use_mock_data`
- `cvp_host` , `cvp_port`
- `cvp_token` 또는 `username/password`
- `ca/cert/key` 관련 TLS 옵션
- 특정 장비 필터(`cvp_device_ids`)
- CORS origin

중요 로직

- CVP_HOST 에 `host:port` 형식이 섞여 들어와도 분리 가능
- OPS_CONSOLE_CVP_LIBRARY_ROOT 가 지정되면 `portable` 배포 폴더 안의 `cloudvision-python-trunk` 를 사용
- OPS_CONSOLE_USE MOCK=true 이거나 CVP 인증정보가 없으면 `demo` 모드
- CVP 인증정보가 있고 `mock`이 `false`이면 `cvp` 모드

9. 경로 설정 계층

`backend/app/core/path_config.py` 는 YAML 파일을 읽습니다. 이 구조 덕분에 텔레메트리 경로와 필드 이름을 코드와 분리할 수 있습니다.

읽는 파일:

- `backend/config/telemetry_paths.yaml`
- `backend/config/field_mapping.yaml`

`telemetry_paths.yaml` 현재 기준

- `devices`
 - `dataset:` `analytics`
 - `path:` `DatasetInfo` / `Devices`
- `vrf`
 - `path:` `Smash` / `vrf` / `vrfIdMapStatus` / `vrfIdToName`
- `bgp`
 - `dataset:` `analytics`
 - `path root:` `Devices` / `<device_id>` / `versioned-data` / `routing` / `bgp` / `config` / `vrf` / `<vrf>`
- `config`
 - `path:` `Config` / `running` / `lines`
- `vlan`
 - `config path:` `Sysdb` / `bridging` / `config` / `vlanConfig`

- svi path: Sysdb / interface / config / eth / vlan / intfConfig

field_mapping.yaml 현재 기준

- BGP
 - asn_field = asNumber
 - asn_value_field = value
 - router_id_field = routerId
 - shutdown_field = shutdown
- VRF
 - name_field = name

10. CVP 연결 계층

backend/app/core/cvp_connector.py 가 CVP 연결을 담당합니다.

주요 책임

- cloudvision-python-trunk 를 import path에 등록
- token 인증 또는 username/password 인증 수행
- 필요 시 CVP 인증서 자동 확보
- GRPCClient 생성
- create_query 를 사용해 dataset/path 질의 생성
- CVP 응답의 key/value를 Python dict / list / 문자열로 정규화

인증 방식

- CVP_TOKEN 이 있으면 토큰 우선 사용
- 없으면 username/password 로 /cvpservice/login/authenticate.do 에 POST
- 응답의 sessionId 를 받아 token처럼 사용

TLS 관련 처리

- CA 파일이 있으면 그 파일 사용
- CA 파일이 없고 insecure 모드가 아니면 서버 인증서를 직접 가져와 context 생성
- insecure=true 이면 인증서 검증을 끈 context 사용

11. 수집 계층 개요

수집 계층의 중심은 backend/app/services/collection_service.py 와 backend/app/collectors/cvp_suite.py 입니다.

역할 분리

- `CollectionService`
 - 전체 refresh 제어
 - source mode 판단
 - progress 상태 관리
 - collector 실행
 - config 파일 저장 호출
 - DB 반영 호출
- `CVPCollectorSuite`
 - 실제 CVP 데이터 수집
- `MockCollectorSuite`
 - 샘플 JSON 로딩

12. CollectionService 상세 설명

`CollectionService` 는 프로그램의 수집 오케스트레이터입니다.

핵심 메서드

- `ensure_seed_data()`
 - 서버 시작 시 최소 1개의 유효 snapshot이 있도록 보장
 - DB가 비어 있으면 refresh 수행
 - 최신 job이 없거나, source가 다르거나, last status가 success가 아니면 refresh 수행
- `refresh()`
 - 실제 전체 snapshot 수집 수행
- `start_refresh()`
 - 백그라운드 thread로 refresh 시작
- `get_progress()`
 - 현재 수집 상태 반환

refresh 처리 흐름

- 시작 시 progress 상태를 `running` 으로 변경
- source에 맞는 suite 선택
- `collect()` 호출
- config 파일 저장
- DB `replace_snapshot()` 호출
- 성공 job 기록
- 실패 시 기존 snapshot은 유지하고 `collection_jobs` 에 failed job만 기록

이 설계의 장점

- 수집 실패 시 기존 정상 데이터가 남습니다.
- 사용자는 완전히 빈 화면이 되는 상황을 피할 수 있습니다.

13. CVPCollectorSuite 상세 설명

CVPCollectorSuite 는 실CVP에서 데이터를 읽어 snapshot dict 를 만듭니다.

최종 반환 구조는 다음 키를 가집니다.

- devices
- bgp
- vrfs
- vlans
- ip_records
- configs

13-1. 장비 수집 로직

devices 는 analytics dataset의 DatasetInfo / Devices 를 사용합니다.

수집 항목:

- device_id
- hostname
- serial
- mgmt_ip
- model
- site
- tags
- last_collected_at

현재 serial 은 원본에서 별도 값을 못 쓰는 경우 device_id 를 대체값으로 넣도록 되어 있습니다.

13-2. VRF 수집 로직

장비별 경로:

```
/Smash/vrf/vrfIdMapStatus/vrfIdToName
```

해석 방식:

- map의 key에서 `vrf id` 를 추출
- map value 내부의 `name` 필드를 `vrf_name` 으로 사용

결과 필드:

- `device_id`
- `hostname`
- `vrf_name`
- `vrf_id`

13-3. BGP 수집 로직

장비별, VRF별 경로:

```
/Devices/<device_id>/versioned-data/routing/bgp/config/vrf/<vrf>
```

읽는 값:

- `asNumber.value`
- `routerId`
- `shutdown`

결과 필드:

- `device_id`
- `hostname`
- `vrf`
- `asn`
- `router_id`
- `shutdown`
- `source_path`

주의점:

- 장비에 VRF가 하나도 안 잡히지 않으면 기본적으로 `default` VRF를 가정하는 `fallback`이 들어 있습니다.
- `asn` 값이 없는 엔트리는 버립니다.

13-4. VLAN 수집 로직

현재 VLAN은 두 단계로 처리합니다.

사용 경로:

```
/Sysdb/bridging/config/vlanConfig
/Sysdb/interface/config/eth/vlan/intfConfig
```

중요한 구현 포인트:

- 상위 `vlanConfig` 경로는 실제 상세 dict가 아니라 자식 key 포인터처럼 오는 경우가 있습니다.
- 그래서 collector는 먼저 상위 key 목록을 읽고,
- 각 key 예: `{"value":200}` 에 대해,
- `/Sysdb/bridging/config/vlanConfig/{"value":200}` 를 다시 조회해 상세 값을 얻습니다.

실제 사용 필드:

- `configSource`
- `configuredName`
- `vlanId`

필터링 규칙:

- `configSource == 'cli'` 인 VLAN만 수집

SVI 설명 처리:

- `intfConfig` 쪽에서 `vlanXXX` 인터페이스 `description` 을 읽어오려 시도
- 해당 데이터가 비어 있으면 SVI 와 `description` 은 X 로 표시

결과 필드:

- `device_id`
- `hostname`
- `vlan_id`
- `vlan_name`
- `svi_name`
- `description`
- `source_path`

13-5. Config 수집 로직

장비별 경로:

```
/Config/running/lines
```

이 경로는 한 줄 한 줄이 linked-list 형태로 저장되어 있습니다.
각 node에는 대체로 다음 정보가 있습니다.

- text
- previous
- next

collector는 이 raw node map을 `reconstruct_config_lines()` 에 넘겨 전체 running-config 문자열로 재조립합니다.

13-6. IP 추출 로직

IP는 별도 telemetry path를 직접 쓰지 않고, 현재 v1 기준으로 running-config에서 추출합니다.

즉, 순서는 다음과 같습니다.

1. Config 수집
2. Config 재조립
3. 인터페이스 IP 파싱

14. Config 재조립 로직

`backend/app/services/config_parser.py` 의 `reconstruct_config_lines()` 가 담당합니다.

작동 방식

- node map에서 `previous` 가 없는 head 후보를 찾음
- 없으면 `next` 관계를 뒤집어 head 후보를 찾음
- head에서 `next` 를 따라가며 순서대로 `text` 를 연결
- 방문되지 않은 orphan node도 뒤에 이어 붙여 손실 최소화

이 로직 덕분에 CVP가 한 줄씩 분리 저장한 running-config를 정상 순서의 텍스트로 복원할 수 있습니다.

15. IP 파싱 로직

`backend/app/services/config_parser.py` 의 `extract_ip_records()` 가 담당합니다.

파싱 대상

- interface 블록 내부의 `ip address` 명령

현재 지원 패턴

- `ip address A.B.C.D/Prefix`

- `ip address A.B.C.D W.X.Y.Z`
- `secondary` 표기가 있어도 주소 자체는 읽음

현재 무시하는 패턴

- `ip address dhcp`
- `ip address negotiated`

인터페이스별 종류 분류

- `Loopback*` -> `loopback`
- `Management*` -> `mgmt`
- `Vlan*` -> `svi`
- 그 외 -> `interface`

VRF 처리

- interface 블록 안에서 `vrf forwarding <name>` 을 만나면 `current_vrf` 를 변경
- 없으면 `default`

결과 필드

- `device_id`
- `hostname`
- `interface_name`
- `ip`
- `address`
- `prefix_length`
- `network`
- `vrf`
- `ip_kind`
- `source=config`

16. Config 파일 저장 방식

`backend/app/storage/config_files.py` 의 `ConfigFileManager` 가 담당합니다.

저장 규칙

- 장비별 폴더 생성: `backend/data/configs/<device_id>/`
- 최신 파일: `latest.cfg`
- 보관 파일: `<timestamp>_<hash10>.cfg`

- hash는 SHA-256

DB에 저장되는 것은 본문이 아니라 메타데이터입니다.
즉, SQLite에는 다음 정보만 들어갑니다.

- device_id
- hostname
- config_hash
- file_path
- collected_at
- line_count

본문을 파일로 저장하는 이유

- DB를 불필요하게 비대하게 만들지 않기 위함
- 최신 파일 접근이 쉬움
- 장비별 파일 백업이 명확함

17. 저장소 계층과 DB 구조

backend/app/repositories/snapshot_repository.py 는 SQLite 저장소를 담당합니다.

주요 테이블

- devices
 - 장비 인벤토리 기본 정보
- bgp_entries
 - 장비별/VRF별 BGP 엔트리
- vrfs
 - 장비별 VRF 목록
- vlans
 - 장비별 VLAN 목록
- ip_records
 - Config에서 파싱한 IP 정보
- config_snapshots
 - 최신 config 파일 메타데이터
- collection_jobs
 - 수집 작업 성공/실패 이력

17-1. snapshot 반영 방식

`replace_snapshot()` 은 전체 snapshot을 최신 상태로 교체합니다.

순서:

- `bgp_entries` 삭제
- `vrf`s 삭제
- `vlan`s 삭제
- `ip_records` 삭제
- `devices` 삭제
- `config_snapshots` 삭제
- 새 snapshot 데이터 삽입
- `collection_jobs` 에 성공 기록 삽입

즉, 이 프로그램의 DB는 **이력형 장기 저장소**보다 **최신 조회용 snapshot 저장소** 성격이 강합니다. 단, `collection_jobs` 만큼은 refresh 시도 이력을 별도로 남깁니다.

17-2. 인덱스

성능 보조를 위해 다음 인덱스가 있습니다.

- `devices.hostname`
- `bgp_entries.asn`
- `bgp_entries.hostname`
- `vrf`s.`vrf_name`
- `vlan`s.`vlan_id`
- `vlan`s.`vlan_name`
- `ip_records.address`
- `ip_records.vrf`
- `ip_records.hostname`

18. 조회 계층 상세 설명

`backend/app/services/query_service.py` 가 실제 조회 결과를 조립합니다.

주요 역할

- overview 응답 구성
- 장비 목록 응답 구성
- config preview 생성
- 목록 API용 record list 생성
- lookup 결과 상태 판정

18-1. overview

반환 값:

- device_count
- ip_count
- bgp_count
- vlan_count
- vrf_count
- config_snapshot_count
- latest_collection_at
- source_mode
- latest_job

18-2. device list

장비 목록은 hostname 기준 정렬됩니다. config_hash 와 config_collected_at 을 함께 내려 UI에서 config 존재 여부를 보여줄 수 있게 합니다.

18-3. config preview

DB에서 file_path 를 읽고, 실제 파일 내용을 열어 API 응답에 content 로 포함합니다. 즉, config preview는 DB 본문이 아니라 파일 본문입니다.

19. 조회 상태값 의미

LookupStatus 는 다음 다섯 가지입니다.

- available
 - 현재 snapshot에서 직접 확인되지 않음
 - UI 의미: **사용 후보**
- in_use
 - 정확히 같은 값이 현재 snapshot에 존재
 - UI 의미: **이미 사용 중**
- review
 - exact match는 아니지만 문맥상 겹침 가능성 존재
 - UI 의미: **검토 필요**
- not_available
 - loopback/mgmt처럼 재사용을 강하게 막아야 하는 경우
 - UI 의미: **사용 불가**
- error

- 입력 형식 오류

20. IP 조회 판정 규칙

IP 조회는 두 종류를 지원합니다.

- 단일 IP 조회
- 대역(prefix) 조회

20-1. 단일 IP 조회

- exact match가 있으면 `in_use`
- exact match가 loopback 또는 mgmt 인터페이스라면 `not_available`
- exact match는 없지만 어떤 기존 network 안에 들어가면 `review`
- 아무것도 없으면 `available`

20-2. 대역 조회

- 동일 network가 존재하면 `in_use`
- 겹치는 network가 있으면 `review`
- 겹치는 것이 없으면 `available`

20-3. 현재 성격

이 로직은 중복 사고 감지기보다는 신규 사용 전 조회기 성격에 맞춰져 있습니다. 즉, 운영 현황을 조회하여 사람이 판단하도록 돕는 것입니다.

21. VLAN 조회 판정 규칙

- VLAN ID와 이름을 둘 다 입력하면 exact 조합 존재 여부를 확인
- 둘의 조합은 없지만 ID 또는 이름 한쪽만 존재하면 `review`
- 둘 다 없으면 `available`
- ID만 또는 이름만 넣은 경우에는 일치 항목이 있으면 `in_use`

22. BGP 조회 판정 규칙

- 동일 ASN이 있으면 `in_use`
- 없으면 `available`
- 결과에는 device, vrf, router_id, shutdown 정보 포함

23. VRF 조회 판정 규칙

- 동일 VRF 이름이 존재하면 `in_use`

- 없으면 available

24. API 구조

REST API는 `backend/app/api/routes.py` 에 정의되어 있습니다.

주요 엔드포인트

- GET `/health`
 - 서버 생존 확인
- GET `/api/overview`
 - 개요 카드 데이터
- GET `/api/devices`
 - 장비 목록
- GET `/api/records/ip`
 - IP 기본 목록
- GET `/api/records/bgp`
 - BGP 기본 목록
- GET `/api/records/vlan`
 - VLAN 기본 목록
- GET `/api/records/vrf`
 - VRF 기본 목록
- GET `/api/lookup/ip?q=...&vrf=...`
 - IP 조회
- GET `/api/lookup/bgp?asn=...`
 - ASN 조회
- GET `/api/lookup/vlan?vlan_id=...&name=...`
 - VLAN 조회
- GET `/api/lookup/vrf?name=...`
 - VRF 조회
- GET `/api/devices/<device_id>/config`
 - 특정 장비의 최신 config preview
- GET `/api/collections/status`
 - refresh 상태 조회
- POST `/api/collections/refresh`
 - refresh 시작

25. 프론트엔드 구조 설명

portable 폴더에는 frontend/dist 만 포함되어 있습니다. 즉, 브라우저가 읽는 build 결과만 있습니다. 원본 개발 구조 기준 UI 핵심은 React/Vite로 작성되었습니다.

개발 기준 주요 파일

- frontend/src/main.tsx
 - React 진입점
- frontend/src/App.tsx
 - 메인 화면 구성
- frontend/src/api.ts
 - API 호출 래퍼
- frontend/src/types.ts
 - 응답 타입 정의
- frontend/src/App.css
 - UI 스타일
- frontend/src/index.css
 - 전역 스타일

portable 폴더에는 build 결과만 있으므로, UI 수정은 원본 소스 저장소에서 해야 합니다.

26. UI 화면 구조

현재 UI는 크게 다음 화면으로 구성됩니다.

- 홈
- IP 조회
- BGP AS
- VLAN
- VRF
- 장비

26-1. 홈

표시 내용:

- 장비 개수
- IP 개수
- BGP 개수
- VLAN 개수
- VRF 개수
- Config 개수

- 읽는 방법 안내
- 상태 라벨 의미 설명

26-2. IP 조회 화면

기능:

- IP 또는 대역 입력 후 조회
- 선택 VRF 입력 가능
- 검색하지 않아도 기본 목록 표시
- 기본 목록은 최대 200건 표시

26-3. BGP AS 화면

기능:

- ASN 입력 후 조회
- 기본 BGP 목록 표시

26-4. VLAN 화면

기능:

- VLAN ID 또는 이름 조회
- 기본 VLAN 목록 표시

26-5. VRF 화면

기능:

- VRF 이름 조회
- 기본 VRF 목록 표시

26-6. 장비 화면

기능:

- 장비 목록 표시
- hostname, mgmt IP, model, site 확인
- 장비 클릭 시 최신 config preview 표시
- 검색 필터 지원

27. refresh 동작 방식

refresh는 config만 새로 받는 기능이 아닙니다. **전체 snapshot 갱신 기능**입니다.

refresh 시 갱신 대상:

- devices
- vrfs
- bgp
- vlans
- configs
- config 기반 ip_records
- config 파일 저장본
- DB 최신 snapshot
- collection_jobs 최신 결과

프론트는 refresh 진행 상황을 progress 카드로 보여줍니다.

현재 step 예시

- queued
- starting
- connect
- device_inventory
- device_details
- snapshot_ready
- config_files
- database
- completed
- failed

28. demo 모드와 live 모드 차이

demo 모드

- sample_snapshot.json 사용
- CVP 연결 불필요
- 기능 시연용

live 모드

- 실제 CVP 접속
- 실제 장비 데이터 수집

- config\live.env 필요

모드 결정 규칙

- OPS_CONSOLE_USE MOCK=true 이면 demo
- 그렇지 않고 CVP credential이 있으면 cvp 모드

29. 정적 프론트 서빙 방식

backend/app/main.py 는 frontend/dist 를 직접 서비스합니다.

동작:

- /assets 는 정적 파일 mount
- / 는 index.html 반환
- 그 외 일반 SPA 경로는 존재 파일이면 그 파일, 아니면 index.html 반환
- /api, /health 같은 백엔드 경로는 프론트로 넘기지 않고 API 라우터가 처리

이 구조 덕분에 브라우저는 별도 프론트 서버 없이 백엔드 서버 한 개만으로 동작합니다.

30. 프로그램의 성격과 한계

이 프로그램은 프로토타입입니다. 현재 강점과 한계는 분명합니다.

강점

- 설치 없이 실행 가능
- CVP 정보를 읽기 전용으로 조회 가능
- 장비, IP, BGP, VLAN, VRF, Config를 한 화면 체계로 볼 수 있음
- 신규 값 사용 전 조회 도구로 적합
- 수집 실패 시 이전 snapshot 보존

현재 한계

- 전체 refresh는 full snapshot 방식이라 대규모 장비 환경에서는 시간이 걸릴 수 있음
- 실시간 streaming 감시 도구는 아님
- IP는 현재 config parsing 기반 v1 로직임
- IPv6는 현재 지원하지 않음
- DHCP/negotiated 주소는 수집하지 않음
- tags 정보는 현재 적극 활용하지 않음
- UI 소스는 portable 폴더에 포함되지 않음
- DB는 장기 이력 분석용 설계보다 최신 snapshot 조회용 설계에 가깝다

31. 대규모 운영 시 고려사항

장비가 많아질수록 다음 개선이 필요할 수 있습니다.

- 병렬 수집
- 장비군별 부분 refresh
- refresh queue/worker 분리
- 이력 DB 분리
- 페이지징된 목록 API
- UI 가상 스크롤 또는 서버 페이지징

즉, 현재 구조는 프로토타입 및 1차 시연에는 적합하지만, 매우 큰 규모의 상시 운영 서비스로 확대하려면 다음 단계 최적화가 필요합니다.

32. 유지보수 포인트

실무적으로 가장 자주 보게 될 파일은 다음과 같습니다.

- `config\live.env`
 - CVP 연결 정보 변경
- `backend\config\telemetry_paths.yaml`
 - 경로 변경 시 수정
- `backend\config\field_mapping.yaml`
 - 필드 키 구조 변경 시 수정
- `backend\data\db\ops_console.db`
 - snapshot DB
- `backend\data\configs\<device_id>\latest.cfg`
 - 최신 config 본문

원본 소스 기준으로는 아래도 중요합니다.

- `backend\app\collectors\cvp_suite.py`
- `backend\app\services\config_parser.py`
- `backend\app\services\query_service.py`
- `backend\app\repositories\snapshot_repository.py`
- `frontend\src\App.tsx`

33. 장애 또는 이상 시 점검 순서

1. `run-live.bat` 가 정상 실행되었는지 확인
2. `config\live.env` 값 확인

3. 브라우저에서 /health 확인
4. /api/overview 응답 확인
5. backend/data/db/ops_console.db 생성 여부 확인
6. backend/data/configs 내부 파일 생성 여부 확인
7. CVP 인증정보 또는 TLS 설정 확인
8. telemetry path 구조 변경 여부 확인

34. 이 프로그램이 하지 않는 일

아래 기능은 현재 범위 밖입니다.

- 장비 설정 변경
- CVP task 실행
- config push
- change control 승인/실행
- 실시간 장애 탐지
- 자동 치유
- 완전한 IPAM 대체
- 완전한 NMS 대체

35. 최종 요약

이 프로그램은 다음 한 문장으로 설명할 수 있습니다.

CVP에 등록된 장비들의 상태와 주요 자원(IP, BGP ASN, VLAN, VRF, Config)을 읽기 전용 스냅샷으로 수집해, 사용자가 웹 화면에서 간단히 확인하고 신규 사용 가능성을 조회할 수 있게 해 주는 portable 운영 보조 콘솔이다.

즉, 이 프로그램의 핵심은 **제어**가 아니라 **조회**, **자동 승인**이 아니라 **운영 판단 보조**, **실시간 감시**가 아니라 **현재 스냅샷 기반 현황 확인**입니다.